

EPaxos Revisited

Sarah Tollman, *Stanford University;* Seo Jin Park, *MIT CSAIL;*
John Ousterhout, *Stanford University*

https://www.usenix.org/conference/nsdi21/presentation/tollman

This paper is included in the
Proceedings of the 18th USENIX Symposium on
Networked Systems Design and Implementation.

April 12–14, 2021

978-1-939133-21-2

Open access to the Proceedings of the
18th USENIX Symposium on Networked
Systems Design and Implementation
is sponsored by



EPaxos Revisited

Sarah Tollman
Stanford University

Seo Jin Park
MIT CSAIL

John Ousterhout
Stanford University

Abstract

This paper re-evaluates the performance of the EPaxos consensus protocol for geo-replication and proposes an enhancement that uses synchronized clocks to reduce operation latency. The benchmarking approach used for the original EPaxos evaluation does not trigger or measure the full impact of conflict behavior on system performance. Our re-evaluation confirms the original claim that EPaxos provides optimal median commit latency in a WAN, but it shows much worse tail latency than previously reported (more than 4x worse than Multi-Paxos). Furthermore, performance is highly sensitive to application workloads, particularly at the tail.

In addition, we show how synchronized clocks can be used to reduce conflicts in geo-replication. By imposing intentional delays on message processing, we can achieve roughly in-order deliveries to multiple replicas. When applied to EPaxos, this technique reduced conflicts by at least 50% without introducing additional overhead, decreasing mean latency by up to 7.5%.

1 Introduction

Consensus-based replication protocols such as Raft [23] and Multi-Paxos [15] play an important role in large-scale datacenter applications. These protocols have traditionally required two round-trip times (RTTs) between machines in order to ensure durability and consistency before returning results to clients. The latency impact of these RTTs is particularly severe in geo-replicated applications, which have replicas in geographically distributed datacenters with inter-datacenter RTTs of 100ms or more.

In recent years there have been numerous proposals for new consensus protocols built around a *fast path* that can complete many operations with a single RTT [16, 21, 24, 30]. Operations that meet certain conditions, such as being commutative with other pending operations, can proceed along the fast path, while operations not meeting the conditions require at least two RTTs.

Unfortunately, most of these proposals, particularly those that exploit commutativity, work best within a datacenter, where RTTs are low [16, 24, 30]. High network delays in a wide-area network (WAN) make it more difficult to exploit commutativity because they increase the window of time during which operations can conflict. As conflicts increase, fewer operations can complete in 1 RTT, which reduces the latency benefit of the protocol.

Egalitarian Paxos (EPaxos) [21] is a commutativity-based consensus protocol that claims to have low latency in a WAN. As with other fast-path protocols, the performance of EPaxos is heavily dependent on workload, as the workload determines the proportion of operations that are able to benefit from the fast path. The original EPaxos evaluation included multiple scenarios for command interference, which plausibly measured best and worse case performance.

However, while replicating the EPaxos paper results, we discovered that the original benchmarking methodology did not trigger, and the metrics chosen did not capture, the full impact of conflicts on EPaxos performance. We reproduced the original EPaxos results, then broadened the evaluation to cover a wider range of potential conflict behavior. We also collected different metrics, which we believe more holistically reflect protocol performance.

Our re-evaluation draws different conclusions about EPaxos than the original evaluation. The original evaluation concluded that EPaxos has optimal commit latency in a WAN, where "optimal" is defined as 1 RTT to a simple majority of replicas. We agree that many operations can achieve this optimal latency, but we show that the percentage of operations completing in 1 RTT varies greatly depending on the workload. In one of our experiments, this percentage was as low as 65%. We also demonstrate that the latency increase for conflicting operations is substantial; even with a safe upper bound on execution latency many conflicting operations experience latency that is more than 4x worse than that of Multi-Paxos.

In the process of evaluating EPaxos more deeply, we discovered two opportunities for improving EPaxos' performance. When EPaxos runs on highly skewed workloads, dependency chains can grow without bound, resulting in tail latencies above 5 seconds or even livelock. We found a simple modification to EPaxos that prevents unbounded dependency growth and ensures a tight upper bound on latency. Second, we improved EPaxos performance using synchronized clocks. We noticed that conflicts occur primarily because different replicas process operations at different times. We modified EPaxos to use synchronized clocks to ensure that all quorum replicas process a given operation at the same time. This reduces conflict rates by at least 50% without introducing any additional latency.

This paper makes the following contributions:

- We show how that the original EPaxos evaluation does not fully capture the impact of conflicts on system performance. The EPaxos benchmarking framework has been used by subsequent papers such as Bipartisan Paxos [30], so we think it is important to publicize these limitations to prevent them from propagating further.
- We re-evaluate EPaxos with an improved benchmarking framework that captures a wider range of possible conflict behaviors. Our evaluation shows that the benefits of EPaxos are narrower and the bad behaviors significantly worse than previously reported. It is not safe to conclude that EPaxos will improve performance for a given applica-

tion without knowing details of the application's behavior and whether tail latency matters.
- We show how synchronized clocks can be used to reduce EPaxos conflict rate and increase its feasibility for a wider range of workloads.

The remainder of this paper is organized as follows: Section 2 provides an overview of the EPaxos protocol. Section 3 describes how we were able to provide an upper bound on EPaxos execution latency. Section 4 describes how we improved EPaxos by taking advantage of synchronized clocks. Section 5 details the differences between our EPaxos evaluation methodology and the original. Section 6 evaluates EPaxos and our improvements. Section 7 discusses related work, and Section 8 concludes.

2 Egalitarian Paxos Overview

EPaxos claims three benefits: low latency for geo-replication, high throughput, and graceful performance degradation in the face of failures. In this paper we focus on the latency benefit: EPaxos can complete most operations in a single wide-area RTT. Consensus protocols typically require two RTTs to complete an operation: one for the client to communicate with one of the replicas, and another to communicate information among the replicas. Most consensus protocols, such as Paxos [15] and Raft [23], have a single leader through which all operations must pass, so for most clients both of the RTTs for an operation will require wide-area communication. In contrast, EPaxos uses a leaderless approach, so clients can communicate with the nearest replica, typically in the same data center. As a result, only the second RTT requires wide-area communication. The cost of the RTT from the client to the local replica is negligible in terms of overall operation latency.

The challenge for leaderless protocols such as EPaxos is to establish a consistent ordering between operations submitted to different replicas. When *interfering* operations (those that are not commutative) arrive concurrently at different replicas, the replicas must somehow agree on an ordering, so that all replicas execute the operations in the same order. In the worst case, reaching agreement will require at least two WAN RTTs among the replicas. However, in many cases a single WAN RTT provides enough information to allow safe execution. For example, one RTT is sufficient to determine whether concurrent operations are commutative; if they are, then the order of execution does not matter. The performance benefit of leaderless approaches depends on how many operations can take the one-RTT *fast path* as opposed to the two-RTT *slow path*.

EPaxos uses *dependencies* to implement its fast path. When an *originating* replica receives an operation from a client, it issues a WAN RTT to collect dependency information from other replicas. The dependencies describe operations that interfere with the new one. If a quorum of replicas returns the same dependencies, then the operation will take the fast path: the originator will be able to analyze the dependencies, determine the ordering, and execute the operation without initiating any more RPCs (in the absence of crashes). If different replicas return different dependencies, then a second WAN RTT is required to propagate the union of the dependencies to the quorum. After the second RTT, the quorum is now in agreement, so the operation is durable.

In EPaxos, durability is defined in terms of dependencies: once a quorum of nodes has persisted an operation and its dependencies, the operation is guaranteed to execute eventually, even in the face of crashes. EPaxos uses the term *committed* to refer to an operation that is durable in this way. At the time an EPaxos operation commits, it has not been executed and its execution order may not even be known. Nonetheless, enough information has been persisted so that the operation can eventually be ordered and executed.

In the remainder of this section we will separately describe the EPaxos commit and execution protocols.

2.1 Commit Protocol

When a replica receives an operation from a client, it assigns that operation to an *instance*. Instances typically contain a single operation, but a replica may choose to batch multiple operations into a single instance. When batching occurs, the dependencies for the instance include all the dependencies for its constituent operations.

Each instance is assigned a globally unique *instance number* consisting of a unique identifier for the originating replica and a sequence number within that replica. Instance numbers provide an ordering among all the instances from a given originator.

In addition to assigning an instance number to each instance, the originating replica also proposes a *dependency set* and *sequence number* for the instance. The dependency set consists of the highest conflicting instance number from each replica (in addition, all preceding instances for the replica are also treated as dependencies). The initial sequence number is chosen as one greater than the largest sequence number in the dependency set. As will be seen below, the sequence number is used to order operations that are mutually dependent. The originating replica derives the dependency set and sequence number for a new instance from all other instances it has knowledge of. The durable storage for each object in the system must include the highest instance number (and sequence number) for each replica that has modified the object.

The first WAN RTT for an instance, called the *PreAccept* phase, begins with these suggestions from the originator. The originator sends the instance, along with its dependencies and sequence number, to a quorum of other replicas. Each replica confers with its state to provide its own suggestion for dependency set and sequence number for the instance, based on all of the instances it knows about. If the peer disagrees on the dependency set or sequence number for the instance, it will merge its determinations with those of the originator. The peer will reply with a dependency set that is the union of the one it received and the one it determined from its state, and a sequence number that is the maximum of the two. The peer persists this information locally and will consider it when

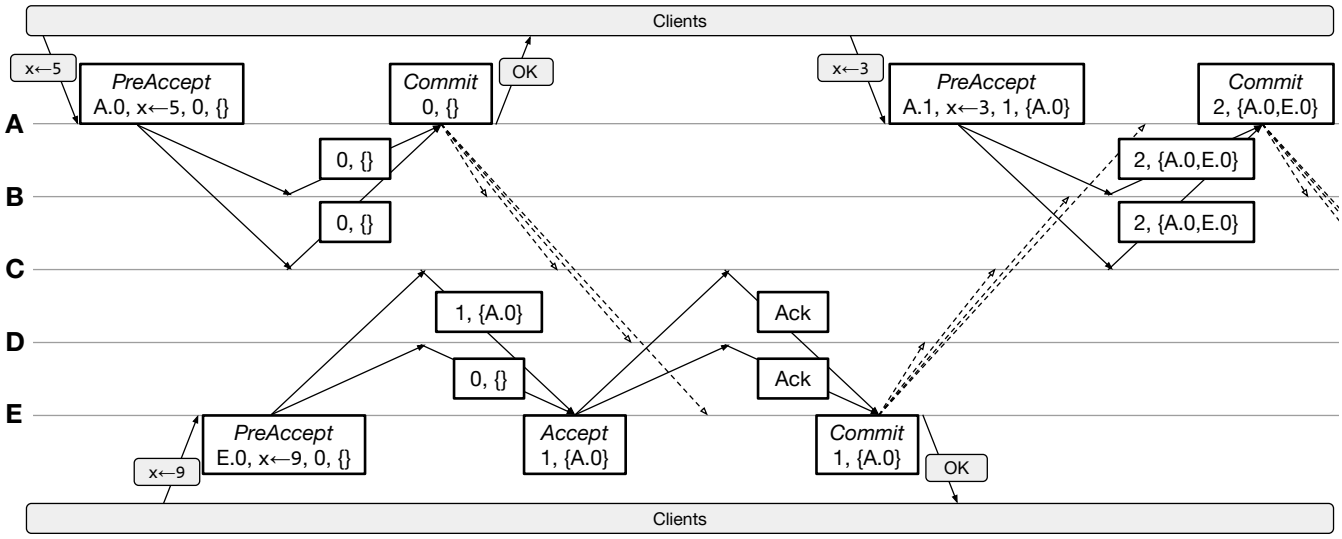


Figure 1: EPaxos commit protocol examples. A, B, C, D, and E are replicas in a key-value store. PreAccept boxes contain the instance number, operation, sequence number, and dependency set for the instance. Subsequent inter-replica messages contain only the sequence number and dependencies. For simplicity, we only show PreAccept and Accept messages to the necessary quorum, and commit messages are indicated by dotted arrows. This figure is adapted from the EPaxos SOSP presentation [18].

processing future PreAccept requests; in replying, the replica promises to reserve that sequence number and dependency set for that instance, until it hears otherwise from the originator.

If the originator receives identical responses from a quorum of peers, it can commit on the fast path. When this happens, it considers the instance committed; it sends Commit messages asynchronously to all other replicas and proceeds to the execution phase discussed in Section 2.2 below. For example, in Figure 1, a quorum of replicas A, B, and C agree on the sequence number and dependencies for instance A.0, so A.0 can commit on the fast path.

If the originator receives conflicting responses, it must issue a second round of RPCs for the instance, called the *Accept* phase. In the Accept phase, the originator merges the sequence numbers and dependency sets from the fast-path quorum's PreAccept replies and sends those to other peers. Because sequence numbers and dependency sets are strictly increasing, the Accept phase will always succeed; each of the recipients of an Accept request has already reserved a lower sequence number and smaller dependency set for the instance, corresponding to an earlier execution order, so it can surely accept higher ones, corresponding to a later execution order. In Figure 1, the PreAccept replies from C and D for E.0 differ, so E merges the responses and commits E.0 after the dependencies and sequence numbers have been replicated to the quorum.

An instance can still commit on the fast path if its originator disagrees with the PreAccept replies, so long as the replies agree with one another. Because sequence numbers and dependencies can only increase, the originator can increase its values for those properties to match the responses, achieving quorum agreement without further communication with other replicas. In Figure 1, instance A.1 can commit on the fast path because

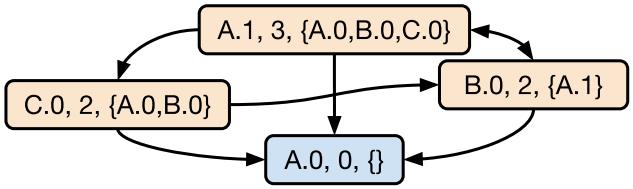


Figure 2: EPaxos execution algorithm example. Each node represents an instance, annotated with its instance number, sequence number, and dependency set. There are directed edges from each node to its dependencies. The nodes are colored by strongly connected component. Traversing the graph yields the execution order A.0, B.0, C.0, A.1.

B and C agree on its sequence number and dependencies. Although A did not initially agree with B and C, A increases its sequence number and dependencies to those agreed on by B and C in order to commit on the fast path.

EPaxos' fast-path quorum size is optimal for common 3- and 5-node cluster sizes. The EPaxos fast-path quorum size is $F + \lfloor \frac{F+1}{2} \rfloor$ replicas, where F is the number of tolerated failures and there are $N = 2F+1$ total replicas. For common 3 and 5 node cluster sizes, EPaxos' fast-path quorum is a simple majority. This is an improvement over the fast-path quorum of prior fast-path consensus protocols, such as Generalized Paxos [16], which has a fast-path quorum size that is always one node larger than that of EPaxos. The EPaxos slow-path quorum size is always a simple majority. EPaxos' quorum sizes are made possible by its crash recovery mechanism. The EPaxos crash recovery protocol is not relevant to the performance-oriented discussions in this paper, and is detailed in the EPaxos paper.

2.2 Execution Protocol

An instance's execution order is determined by creating and traversing a graph of its dependencies. This implies that an in-

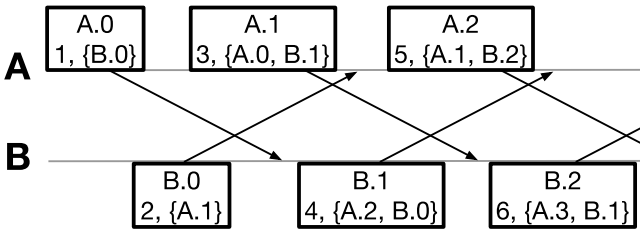


Figure 3: Example of EPaxos infinite dependency chains. Replicas A and B originate instances on the same hot key. In this example, there are three replicas in the system, but A and B are closer together and use each other as their quorum. The arrows indicate PreAccept messages and their timing. Each box contains the instance number, final sequence number, and final dependency set for the instance.

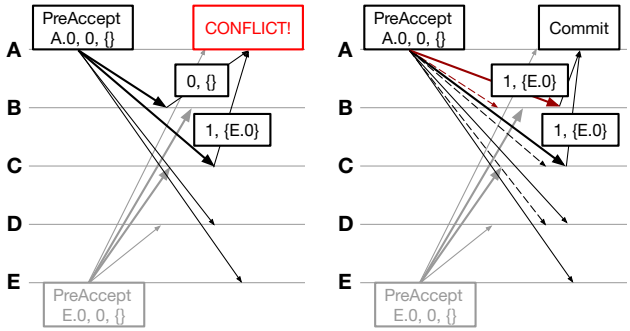
stance must wait for all of its transitive dependencies to commit before executing. In the dependency graph, nodes are instances and directed edges point from instances to their dependencies. As discussed in Section 2.1, if an instance is dependent on R.n, it is implicitly dependent on R.0 through R.(n-1), and R.0 through R.(n-1) would be added as nodes in the graph. The graph can be pruned to contain only those instances that have not executed and whose operations actually interfere with those of the current instance. The graph contains not just an instance's direct dependencies, but also the transitive closure of dependencies. In order to create this graph, and before it can be traversed, all transitive dependencies must have committed so that their sequence numbers and dependency sets are final.

Once an instance's dependency graph is final, it can be traversed to determine execution order. First, the graph is separated into strongly connected components – groups of nodes within which there is a path from each node to every other node. The strongly connected components are executed in reverse topological order. Within a component, instances are executed in order of increasing sequence number, with ties broken in a deterministic manner (e.g. by replica id). Figure 2 provides an example of the execution algorithm. In the example, A.0 forms its own strongly connected component, since it does not have any dependencies. Instances A.1, B.0, and C.0 form another strongly connected component. By reverse topological order, the strongly connected component containing A.0 will be executed first. Within the remaining strongly connected component, the nodes are executed in order of increasing sequence number, breaking the tie between B.0 and C.0 by replica id.

3 Bounding Dependency Chains

Execution latency is unbounded in the original EPaxos protocol because dependencies may chain recursively and an instance cannot execute until all of its transitive dependencies have committed. The EPaxos paper discussed this problem, but the problem did not occur with the workloads used for the original evaluation.

An instance's transitive dependency set can continually grow if new interfering instances are originated before all replicas become aware of prior instances. If this happens, the instance will wait on more and more recently added transitive dependencies to commit, and will be unable to execute. Figure 3 provides an example of this. In the example, a pair of replicas, each of which is in the other's quorum, continually originates new instances on some hot key. Although A.0 is originated first, B originates B.0 before B becomes aware of A.0. For this reason, A.0 becomes dependent on B.0. Similarly, B.0 becomes dependent on A.1, A.1 becomes dependent on B.1, B.1 becomes dependent on A.2, and A.2 becomes dependent on B.2. A.0 is now transitively dependent on B.2. With this behavior, this pattern could continue infinitely, causing execution livelock in the worst case. In practice, this problem causes high execution latency for skewed workloads, as demonstrated in Section 6.4.



(a) Unmodified EPaxos **(b)** With TOQ

Figure 4: EPaxos PreAccept phase example, with and without TOQ. Replicas A and E originate interfering instances A.0 and E.0 concurrently. Without TOQ, the responses from B and C differ for A.0. With TOQ, replica C delays processing the PreAccept for A.0 until the time that C processes it, avoiding conflict.

We are able to bound execution latency by pruning those transitive dependencies that will necessarily be executed after the current instance. Given two instances A and B, the EPaxos execution algorithm states that if B has A as a dependency and B has a higher sequence number than A, then B is serialized after A. The proof of Theorem 5 in the EPaxos proof of correctness [19] proves that, given this serialization, B will necessarily execute after A. Therefore, B can be pruned from A's execution graph. Any new interfering transitive dependency C of B will also necessarily execute after A, because C has a higher sequence number than A and has A in its dependency set, so it can be pruned as well. For example, in Figure 3, B.0 has a higher sequence number than A.0 and has A.0 as a dependency, so B.0 and its yet unknown transitive dependencies (A.1, and A.1's transitive dependencies) can be pruned from A.0's execution graph. With our modification, execution delay is bounded by approximately 3 WAN RTTs. Appendix A.1 describes the bound in more detail.

4 Using Synchronized Clocks to Reduce Conflict Rate

In this section we present a novel approach that reduces conflicts in EPaxos by taking advantage of synchronized clocks. We call this approach Timestamp-Ordered Queueing, or TOQ.

The benefits of EPaxos, and any commutativity-exploiting consensus protocol, depend on minimizing interference between non-commutative operations. By synchronizing the physical clocks of all replicas in the cluster, we are able to order instances so that all replicas process them in the same order, which can eliminate interference even for non-commutative operations. Our approach to ordering is an enhancement; out of order messages or poorly synchronized clocks may result in less conflict mitigation, but do not hinder correctness. We implemented our approach on EPaxos, but we believe it can be generally applied to any fast-path consensus protocol. (We leave this for future work.)

Conflicts occur when replicas learn about instances in different orders. In EPaxos, replicas first learn about an instance they did not originate through its PreAccept message. When the order in which a fast-path quorum processes interfering PreAccept messages differs, the quorum disagrees on the dependency information for those instances, causing them to take the slow path. Figure 4a illustrates an example of this. In the figure, replica A sends a PreAccept message for A.0 at the same time as replica E sends a PreAccept message for E.0. In the example, the fast-path quorum for instances originated from A consists of B and C. Replica B processes the PreAccept messages in the order (A.0, E.0), while replica C processes them in the order (E.0, A.0). Because of this difference in ordering, B and C respond to A's PreAccept with different sequence numbers and dependency sets, causing A.0 to take the slow path.

In the original EPaxos protocol, the order in which a replica learns about instances is determined by the one-way delay between the replica and the originators of the instances. In Figure 4a, the delay between A and B is shorter than that between E and B, and the delay between E and C is shorter than that between A and C. The difference in the one-way delays from the originator to each replica in the fast-path quorum determines whether an instance is ordered before or after instances that interfere with it. By the time a PreAccept message is processed by the furthest replica from its originator, the responses from the closer replicas are stale.

To reduce conflict rate, closer replicas can delay the time at which they process a PreAccept message until the time at which the furthest replica would process it. In Figure 4a, by the time C processes A.0, B and C have both processed E.0. If B were to respond at the same time as C, B and C would respond with the same dependency information. In Figure 4b, replicas delay processing PreAccepts until the time at which the furthest replica processes them. If B delays processing A.0 until the time that C processes it, both B and C have seen E.0 before A.0, and A.0 can commit on the fast path.

4.1 Design

Because replicas cannot intuit the time at which the furthest replica from the originator will process a PreAccept message, each PreAccept message includes a *ProcessAt* timestamp. Each replica will estimate the one-way delay from itself to all other replicas. The ProcessAt time will be the sum of the time at which the PreAccept is sent and the one-way delay from the originator to the furthest replica from the originator.

Each replica sorts PreAccepts in a priority queue based on the ProcessAt time of the PreAccept. Highest priority is given to lower ProcessAt times. An element is dequeued and processed once its ProcessAt time has passed. Because each replica receives the same ProcessAt time for a given instance, if PreAccepts are processed in order of their ProcessAt times then conflicts should not occur.

In order for instances to be totally ordered, a replica must also delay assigning dependency information to instances that it originates. To illustrate why this is necessary, we will look at an example of how conflict can occur when replicas do not delay their own instances. Hypothetically, if replicas A and B originate interfering instances A.0 and B.0 concurrently. A would process the instances in the order (A.0, B.0), and B would process the instances in the order (B.0, A.0). Instances will be processed out of order if a replica does not delay those instances it originates.

To delay the time at which a replica assigns dependency information to its own instances, we leverage the fact that the originator of an instance need not agree with the replies to its PreAccept messages, so long as the replies are unanimous. With TOQ, all PreAccept messages are sent out with sequence number 0 and empty dependencies. The originator adds its own PreAccept messages to its PreAccept priority queue and processes them at their ProcessAt time, consistent with the time they should be processed by the other replicas in the cluster. This means that an originator will not add its instances as dependencies of other instances until other replicas would do the same.

To maintain correctness, we needed to slightly modify the commit protocol for EPaxos. In unmodified EPaxos, an instance can commit on the fast path if the PreAccept replies are unanimous, even if the originator disagrees. With TOQ, since the originator modifies the dependency information for its own instances after it sends the PreAccept messages for them, it is possible that the originator would assign a greater sequence number or dependencies than would its quorum, even if the quorum agrees with one another. Instead of allowing the originator to disagree with the rest of the quorum no matter what, the replica can now disagree with the quorum only if the dependency information of the quorum is greater than that of the originator. Lemma 1 of the EPaxos proof of correctness [19], "EPaxos replicas commit only safe tuples," depends on a fast-path quorum recording the same tuple for any instance that commits in the PreAccept phase. With this modification, we ensure that this holds true.

4.2 Sync Groups

In practice, there is a tradeoff between reducing conflict rate and increasing the minimum latency for each instance. So far, we have described TOQ with the approach of synchronizing all PreAccept messages to all replicas in the cluster. To do this, the ProcessAt time for each PreAccept is the estimated time the PreAccept would be processed at the furthest replica. However,

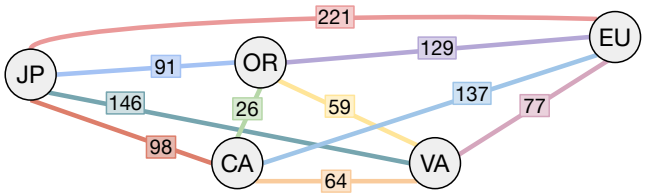


Figure 5: Round-trip times (in ms) between replicas in different locations in our test topology, as measured by the `ping` application. Replicas are located in Virginia (VA), California (CA), Oregon (OR), Japan (JP), and London (EU).

this increases the minimum possible latency for an instance. To illustrate this, we can use back-of-the-envelope calculations based on the RTTs from our test cluster, shown in Figure 5. The figure contains RTTs, and we estimate one-way delay to be half of RTT. Without PreAccept ordering, the minimum latency for an instance that originated in Japan would be 98ms, which is the RTT to the furthest replica in its quorum, California. With PreAccept ordering to the whole cluster, California would delay processing a PreAccept from Japan by the difference in the one-way delay between Japan and California and Japan and London, which is 61.5ms. This increases the minimum latency for an instance originated in Japan by more than 50%.

Another option would be to synchronize message ordering to the furthest replica in each originator's quorum. This approach has the same minimum latency as does unmodified EPaxos. However, syncing to just the quorum is less effective when it comes to reducing conflict rate. Each originator's quorum handles all of the originator's messages in the same order, but may handle messages originating from other replicas in different orders, leading to discrepencies in the dependency information in PreAccept replies.

A hybrid approach is to synchronize message ordering among a quorum *union*. This quorum union contains all replicas included in another node's fast-path quorum. With this sync group, conflicts should occur only when the originator assigned greater dependency information to an instance than did the remainder of its quorum. The benefits of this approach differ greatly depending on the topology; for some topologies, the quorum union might contain the entire cluster.

5 Capturing the Impact of Conflicts

The evaluation approach in the EPaxos paper used workloads that did not trigger the full range of conflict behaviors, and it chose metrics that did not measure the full impact of conflicts. As a result, its conclusions about performance are overoptimistic. In this section we discuss the implications of the original EPaxos evaluation and introduce the changes we made in our evaluation.

5.1 Commit vs. Execution

The EPaxos paper measures latency in terms of commit time, not execution time. Specifically, latency is measured as the time from initiation of an operation on a client until receipt of a commit acknowledgment from the local replica. This approach was chosen because a commit guarantees that an operation is durable and will eventually execute; thus clients need not wait for execution. However, this argument applies only to operations that return no results or errors, such as blind writes (blind writes are the only operation implemented in the EPaxos benchmarks). Commit latency is not globally sufficient; if a client depends on information returned by an operation, then it must wait for execution. For example, the write operations in both Zookeeper [1] and Redis [2] return results. Furthermore, if an operation can result in an error (such as attempting a bank account withdrawal without sufficient funds, or attempting to set a value in a table that has been dropped) then the client must wait for execution to complete. Furthermore, all reads must wait for preceding conflicting writes to execute. Thus, we think it is likely that most operations will require clients to wait for execution, and we argue that EPaxos latency should be measured with execution time, not commit time.

5.2 Access Patterns

One of the challenges in evaluating a protocol like EPaxos is that its performance depends on the application workload. The EPaxos benchmarks use a key-value store with an access pattern based on a single hot key. Each reference is made either to the hot key, or to a unique key that is never referenced by any other operation. Conflict rates are controlled by adjusting the fraction of references to the hot key (either 0%, 2%, or 100% in the EPaxos paper latency evaluation). This approach has two limitations. First, having all interference on a single key understates the impact of conflicts, since there can never be more than a single strongly-connected component in the dependency graph. Second, the EPaxos evaluation conflated the fraction of references to the hot key with the rate of conflicts in the EPaxos commit protocol; because conflict rate is a product of many additional factors, such as system load, the actual conflict rate is considerably less than the fraction of references to the hot key. For example, the workload labeled '100% interference', which was claimed to be worst case, produced actual conflict rates as low as 0%.

For our evaluation we used a Zipfian distribution [11], which is generally accepted as an approximation of real workloads. For example, both Memcached [6] and Facebook's social graph database [4] experience highly skewed workloads that approximate Zipfian. No single value of the Zipfian skew parameter is representative of all applications, so we made measurements with multiple values ranging from lightly skewed (.6) to highly skewed (.99). This covers the ranges used by other benchmarks such as Linkbench [3] and YCSB [12] and allows us to observe how application characteristics affect EPaxos performance.

We also used a mix of read and write operations in our evaluation, whereas the EPaxos paper used only writes. Read operations have different behavior from writes: they do not interfere with other reads, but they do interfere with writes. Thus the mix of reads and writes affects the conflict rate.

5.3 Load

The approach to load generation in the EPaxos paper minimizes opportunities for command interference, which results in over-

optimistic conclusions about latency. First, the paper measures latency only at low system load; higher loads will result in more conflicts, which will increase latency. Furthermore, load is generated with a collection of closed-loop clients, each of which issues a continuous stream of back-to-back operations, with exactly one operation outstanding at all times. Load is controlled by varying the number of clients. We found that this produced a caravan effect, where large groups of operations passed through a replica at once, followed by gaps with no operations. This behavior is unlikely to occur in real applications, and results in an unvarying system load, while real systems are likely to experience bursts. Even systems that use server proxies for admission control still experience variation in load when they are not at maximum capacity. Finally, back-to-back operations result in uneven loading across clients: clients that are more centrally located have higher throughput than those further away, since throughput is inversely proportional to RTT with this approach.

For our measurements we used higher loads (typically 80% of the maximum sustainable load) and we used a Poisson process to schedule new operations, which results in uneven loading that better approximates real workloads. A Poisson arrival process is open loop, which means that care must be taken to choose an arrival rate that does not exceed the system's capacity. If a Poisson arrival process results in system overload, queues will grow without bound, the system does not reach a steady state, and performance measurements are unlikely to be meaningful. We monitored queue lengths in all of our experiments and chose arrival rates for which queue lengths remained bounded. In addition, we added a cap on the total number of outstanding operations for each client (the cap was never reached in our experiments).

5.4 Throughput

There are several confounding factors in the EPaxos throughput measurements, which make it difficult to determine whether the results reflect fundamental properties of the protocols or artifacts of the experimental setup:

- The throughput experiments were run in a LAN, not a WAN. WAN configurations are likely to have higher conflict rates because longer network delays increase the window for conflicts, leading to lower throughput.
- There were different approaches to generating load for different experiments, which led to inconsistent results. With one method, EPaxos '100%' has 12% higher throughput than Multi-Paxos, and with the other it has 27% higher throughput.
- The evaluation of batching (many operations per instance) does not consider the impact of batching on WAN latency.
- The evaluation only considers perfectly even load balancing across clients. Throughput would likely decrease if clients in different locations issued operations at different rates. Correcting for this with load balancing amongst the replicas would negate the latency benefits of EPaxos.
- Optimizations to Multi-Paxos were not considered. For example, the Multi-Paxos leader can hold a read lease for all objects.

5.5 Summary

Evaluation of EPaxos in Section 6 differs from the original EPaxos evaluation in the following respects:

- We measure execution latency, instead of commit latency.
- We use a Zipfian distribution for choosing keys to access, instead of a single hot key.
- We use a mixture of read and write operations, instead of all writes.
- We measure latency at about 80% of maximum throughput, instead of a lightly-loaded system.
- We use Poisson arrival rates, instead of back-to-back operations.
- We measure throughput in the same configuration used for latency measurements, instead of using a LAN configuration with batching.
- We measure a large number of different configurations with different write rates and Zipfian skew factors, instead of 3 hot fractions.

6 Evaluation

We had two goals in our EPaxos evaluation. First, we wanted to see whether our changes to the EPaxos evaluation would change the conclusions about EPaxos performance relative to Multi-Paxos. Our new evaluation confirms the original conclusion that many operations can achieve optimal latency. However, only operations that experience no interference can achieve optimal latency, and we demonstrate that conflict rate varies greatly by application (in our experiments, we measured conflict rates as high as 35%). Additionally, we show that conflicting operations experience significantly higher than optimal latency (more than 4x that of Multi-Paxos, even with our improvements).

Our second goal was to evaluate the effectiveness of TOQ. We found that TOQ is effective at reducing conflict rate, reducing EPaxos latency.

6.1 Experimental Setup

When evaluating EPaxos, we aimed to use an experimental setup as similar to that of the authors as possible. Our analysis is a fork of the EPaxos paper evaluation [20] and is available on GitHub [28]. We used Google Cloud servers in Virginia (VA), California (CA), Oregon (OR), Japan (JP), and London (EU). We used the `n1-standard-8` machine type with the Ubuntu 18.04.5 LTS operating system.

Unless otherwise specified, all experiments mitigate dependency chaining using the approach described in Section 3. EPaxos and Multi-Paxos replicate all operations, including reads. Workload does not impact Multi-Paxos, so we used a workload with a Zipfian skew factor of .9, 50% writes, and 1 million unique keys. Since any reference point is sufficient to see how conclusions change, we compare EPaxos against only Multi-Paxos.

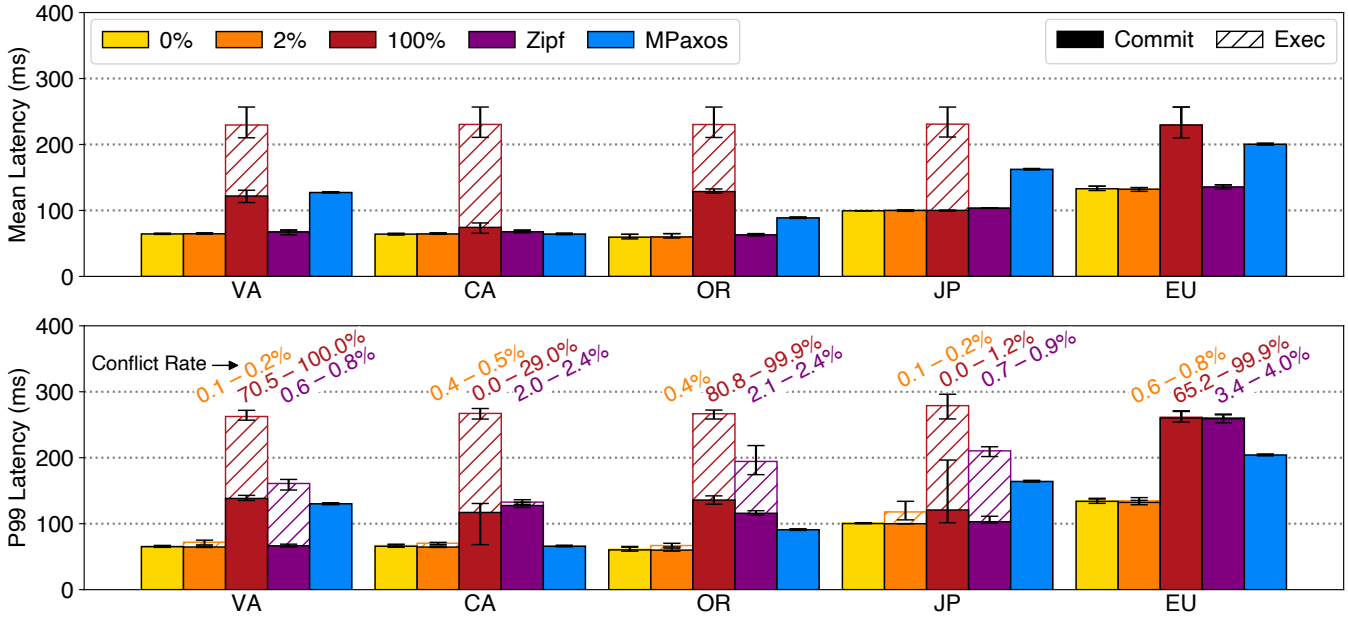


Figure 6: Comparing client-perceived commit and execution latency for EPaxos across a variety of workloads at each of the five sites. 10 clients at each site issue operations, all of which are writes, with outstanding operation at a time. This leads to a throughput of 500 operations per second for Multi-Paxos, 230 operations per second for '100%' and 650 operations per second for the '0%', '2%', and Zipfian workloads. All Multi-Paxos clients issue operations to the leader, which is in California; EPaxos clients issue operations to the replica in the same datacenter. The Zipfian distribution has a skew parameter of .9, 50% writes, and 1 million unique keys. We ran each experiment 5 times, capturing metrics for 60 seconds at steady-state. We show the average latency across the 5 trials, with error bars indicating the minimum and maximum latency of the trials. The range of conflict rates from the trials, measured on the servers, appears above the bar corresponding to the experiment.

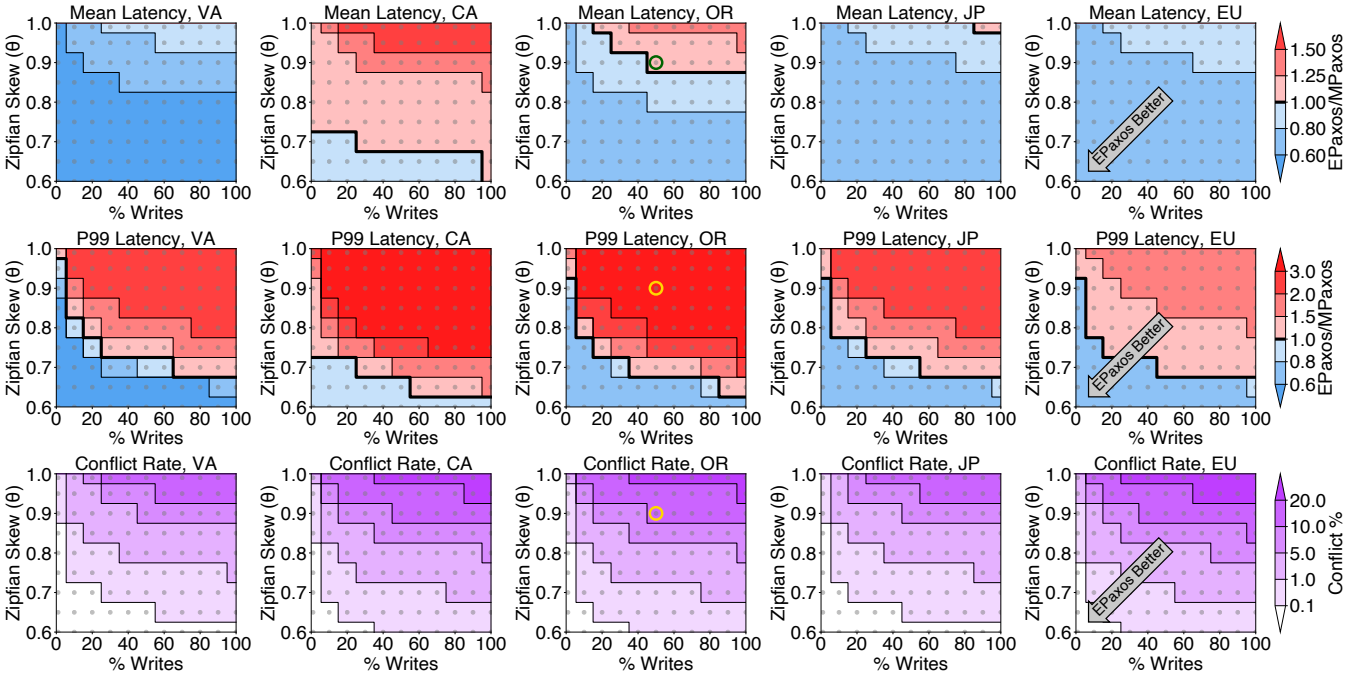


Figure 7: Comparing EPaxos and Multi-Paxos execution latency across a spectrum of Zipfian skew parameters and read-write ratios. Each grey dot corresponds to a workload on which EPaxos was tested. We divide the latency for each EPaxos experiment by Multi-Paxos' latency. Different fill colors represent different ranges of performance; for example, the darkest red color in the top graphs indicates that EPaxos mean latency was at least 1.5x that of Multi-Paxos. All workloads have 1 million unique keys, and all experiments were run at a throughput of 11,000 operations per second, which is 80% of the maximum throughput of EPaxos with a skew parameter of .99 and 100% writes. The circled point indicates the workload used for Figure 8.

We initially ran our experiments for both 3 and 5 replicas, but decided to only include measurements for 5 replicas because performance degraded similarly for both cluster sizes as we modified the experimental methodology. TOQ does not provide a benefit for 3 replicas because conflicts do not occur with a quorum size of 2 (the originator can simply agree with the first replica to respond).

We do not enable *thrifty* – a mode of operation in which PreAccept and Accept messages are only sent to the necessary quorum. We chose to disable thrifty because we wanted our results to be as favorable to EPaxos as possible. As detailed in Appendix A.2, thrifty increases EPaxos conflict rates, which makes latency worse. Thrifty provides the same throughput benefit for EPaxos and Multi-Paxos (about 15%), so enabling thrifty does not impact conclusions about throughput. Furthermore, TOQ is not compatible with thrifty. While decreases in conflict rate due to TOQ are much larger than those from disabling thrifty, we wanted it to be clear in our comparison that any decrease in conflict rate could be attributed to TOQ rather than from disabling thrifty.

6.2 Execution Latency

Our first experiment reproduces the basic commit latency results from the EPaxos paper (see Figure 6); we used the configuration from the EPaxos paper, without any of the changes described in Section 5. We report mean latency instead of median for reasons that will be discussed below. The results are very similar to those in Figure 4 of the EPaxos paper, except that we measured a lower 99th percentile latency for the '2%' workload. This difference does not impact any of our conclusions.

Figure 6 has three additional measurements not present in the EPaxos paper. First, it includes execution latency as well as commit latency. When conflict rates are low, execution latency is indistinguishable from commit latency. However, with high conflict rates, the execution latency can be more than 2x the commit latency. For the pathological '100%' workload, EPaxos execution latency is considerably worse than Multi-Paxos at every site, both at the mean and the 99th percentile. Second, we also measured a Zipfian access pattern; even at low load (about 4% of maximum possible throughput for this experiment), 99th percentile latencies for the Zipfian workload are significantly higher than for the '2%' workload. 99th percentile execution latency under the Zipfian workload is worse than Multi-Paxos at every site. Third, we measured conflict rate on the servers. Primarily due to low load, conflict rates for the '2%' workload were as low as 0.1% and conflict rates for the '100%' workload were as low as 0%.

For the remaining experiments, we applied the evaluation changes discussed in Section 5. One of the challenges in benchmarking EPaxos is that performance varies drastically depending on the workload. It is not difficult to find arguably realistic workloads indicating that EPaxos is either very good or very bad. Thus, we don't believe it is possible to evaluate EPaxos fairly with a few discrete workloads. Instead, we ran each of the remaining experiments over a large range of workloads, typically a full grid with 9 choices of Zipfian skew and 10 choices of read-write ratio. We then plot the data to show how certain system properties vary across the workloads. A potential EPaxos user can use these plots to determine whether EPaxos would provide performance benefits for their particular application. We tried to vary the workload enough to cover the full range of conflict behaviors likely to be experienced by real applications. The experiments were all run at 80% of the maximum throughput of the workload with the lowest maximum throughput. A consequence of keeping throughput consistent for all experiments is that the system is underloaded for many of the workloads. Appendix A.3 contains measurements showing that conflict rate and latency both increase as throughput increases.

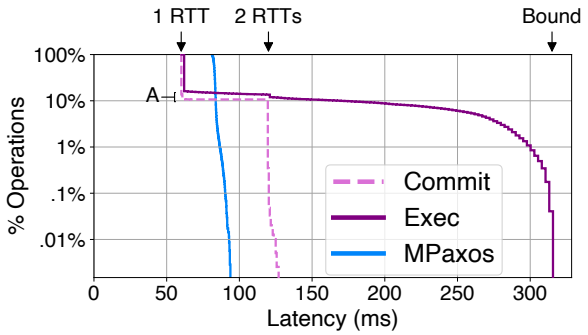


Figure 8: Complementary CDF of execution and commit latency. A point (x,y) indicates that a fraction y of instances had latency greater than x. The data was measured on clients at Oregon on a workload with 50% writes, Zipfian skew 0.9, and a load of 11,000 operations/sec. 'A' indicates the area of instances that experience execution delays despite taking the fast path.

Figure 7 compares the execution latency of EPaxos and Multi-Paxos across the grid of workloads. EPaxos has better mean latency for almost all of the workloads (except for clients in California, which can reach the Multi-Paxos leader without a WAN RTT). On the other hand, EPaxos has worse 99th percentile latency for almost all workloads. EPaxos cannot offer lower latency than Multi-Paxos if there is a conflict in the PreAccept phase, since this results in a second WAN RTT. For EPaxos P99 latency to be better than Multi-Paxos, less than 1% of operations must encounter conflicts; the bottom graphs in Figure 7 show that few workloads have this behavior. In addition to extra RTTs caused by conflicts, EPaxos can also incur execution delays while waiting for dependent operations to commit; Muilti-Paxos does not incur these delays.

Figure 8 exposes several interesting aspects of commit and execution latency:

- Most instances finish execution in one WAN RTT.
- The median latency for both commit and execution will be one RTT unless conflicts approach 50%, which never happened in our experiments. Thus, median latency reflects only network topology, not protocol response to workload.
- Instances that take more than one RTT for execution trend rapidly towards the worst-case delay for dependencies to commit. ('Bound' in the figure.)
- Execution latency is rarely in the vicinity of two RTTs:

if there is a conflict, there will probably be a significant dependency delay.

- A significant fraction of instances that take the fast path experience dependency delay. (This is indicated by the area between the lines at point 'A' in the figure.)
- The worst case performance penalty from EPaxos is greater than the best case performance gain; EPaxos fast path latency is 30% better than Multi-Paxos latency, while the slow path ranges from 40% to 240% worse.

Conclusions about EPaxos latency are almost entirely determined by the choice of metric. The original evaluation used median commit latency as the primary metric. This is the most favorable choice for EPaxos: median latency for EPaxos is independent of application behavior, identical for commit and execution, and uniformly better than Multi-Paxos. Tail latency is often used as a metric in both industry and academia; 99th percentile tail latency is also almost independent of application behavior for EPaxos, and it is significantly worse than Multi-Paxos, especially for execution. Workload differences tend to manifest themselves primarily in the range between the 50th and 99th percentiles, where most metrics will overlook them. We chose to use mean latency instead of median in our graphs, since it reflects some of these workload differences.

6.3 Benefits of TOQ

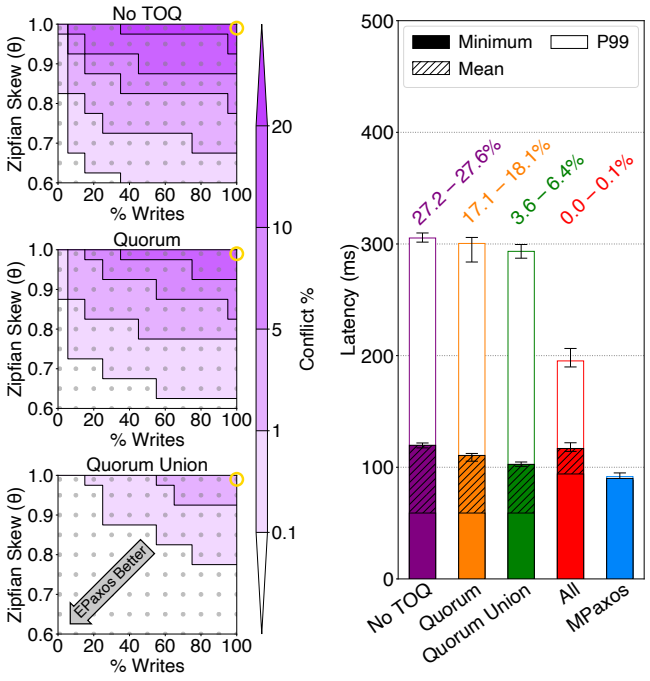
In order to evaluate TOQ we installed the Huygens [10] clock synchronization software to synchronize the clocks of all replicas in the cluster. This resulted in server clock offsets between 20µs and 1ms. The original Huygens evaluation measured clock offsets within 10µs; we believe that we experienced higher clock offsets because of differences in our environments (WAN vs. LAN and virtual machines vs. bare metal).

Figure 9 shows the benefits of TOQ. Quorum sync reduces conflict rates by at least 50% across locations. This reduces mean latencies, which improves the benefits of EPaxos relative to Multi-Paxos. Quorum sync has the advantage of not impacting minimum latency (latency is already limited by the most distant peer in the quorum; quorum sync delays the other peers in the quorum so they process PreAccept messages at the same time as the most distant peer).

Quorum union sync introduces a performance tradeoff. It reduces conflicts by at least 85%, which improves worst-case latency. However, it increases minimum latency. For example, quorum union sync delays Japan's PreAccept messages until they would reach Virginia, which adds about 25ms to the minimum latency for operations issued by clients in Japan. This does not impact the minimum latency for Oregon in Figure 9b, but the impact on Japan can be seen in Figure 20 in the Appendix. Syncing to all replicas completely eliminates conflict, but increases minimum latencies so much that it makes this option unattractive.

6.4 Unbounded Dependency Chains

Our focus on execution latency, combined with Poisson arrivals, exposed the problem of unbounded dependency chains



(a) Conflict Rate **(b)** Latency

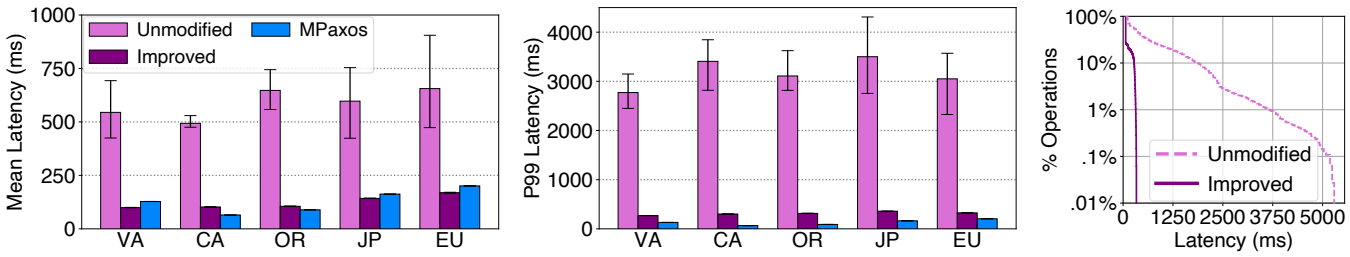
Figure 9: Conflict rate and execution latency for EPaxos with and without TOQ. Figure (a) compares conflict rates for different TOQ sync groups. Figure (b) compares Mean and P99 execution latency for the circled workload (Zipfian skew of .99 and 100% writes). 'Minimum' indicates the minimum possible latency for an operation given the protocol and sync group. Other than TOQ, the experiments for this graph were run with the same methodology as Figure 7. Error bars and conflict rates are shown as in Figure 6. The quorum union consists of Virginia, California, and Oregon. For measurements of sites other than OR, as well as the latency graph for a workload with Zipfian Skew .8 and 50% writes, see Appendix A.4.

discussed in Section 3. Figure 10 shows that EPaxos execution latency can be much higher than Multi-Paxos when it does not protect against infinite dependency chains. Before we implemented our optimization, we observed execution latencies as high as 5 seconds under high-skew workloads. Livelock occurred with the pathological '100% conflict' workload when we removed the cap on outstanding operations from our Poisson scheduler (the cap eventually prevents new operations from entering the system, so the existing operations can complete).

Figure 10c shows that our fix reduces tail latency significantly. The benefits are most noticeable for write-heavy workloads that are highly skewed. However, workloads that generally experience low conflict can still experience bursts of chained dependencies, leading to latency spikes that are surprising and difficult to diagnose. Section 6.6 below shows that that dependency chain optimization also has a significant impact on throughput.

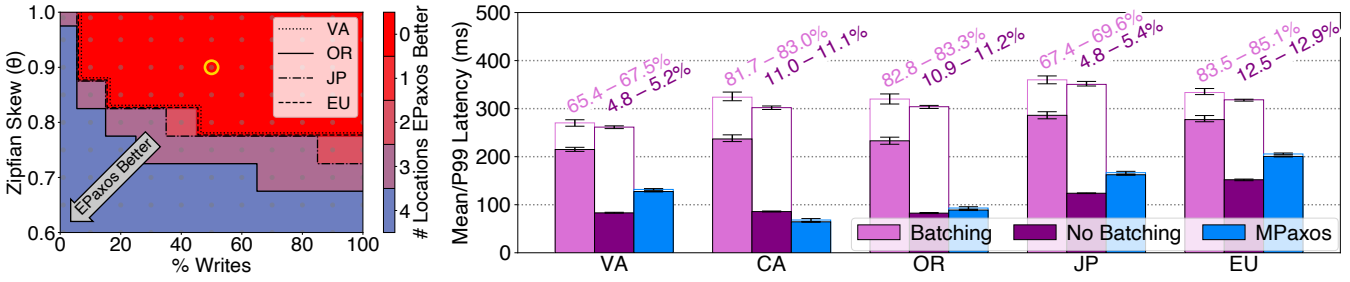
6.5 Latency Cost of Batching

The EPaxos paper demonstrated that batching operations into instances increases the throughput of EPaxos 9x and Multi-



(a) Mean Latency **(b)** P99 Latency **(c)** Complementary CDF

Figure 10: Latency impact of unbounded dependency chains for skewed, write-heaving workloads. All operations are writes, and the Zipfian experiments used a skew parameter of .99. These experiments were run at a throughput of 4000 operations per second, which is 80% of the maximum possible throughput of unmodified EPaxos. Figures (a) and (b) compare Mean and 99th percentile execution latency for EPaxos with out without an upper bound on execution latency. Error bars are shown as in Figure 6. Figure (c) is a complementary CDF showing the fraction of EPaxos operations with latency higher than a given value, with and without bounding dependency chains. Data is shown only for clients in OR; other sites are similar.



(a) Mean Latency **(b)** Circled Workload

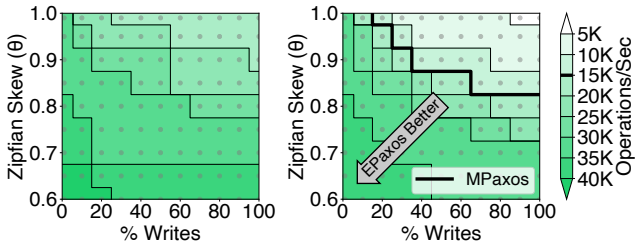
Figure 11: EPaxos execution latency and conflict rate with batching enabled. Figure (a) compares mean latency with batching enabled between EPaxos and Multi-Paxos for a variety of workloads. The lines indicate the workloads for which EPaxos mean latency in that client location is roughly equivalent to Multi-Paxos mean latency; above the line EPaxos latency is worse, and below it Multi-Paxos mean latency is worse. The fill colors indicate the number of client locations that experience a lower latency with EPaxos than Multi-Paxos for the given workload. The circled point indicates the workload in Figure (b): a Zipfian distribution with a skew parameter of .9, where 50% of operations issued by clients are writes. Figure (b) error bars and conflict rates are shown as in Figure 6.

Paxos 5x, but EPaxos batching significantly increases latency because it causes nearly all operations to take the slow path. This is because, with batching, an operation conflicts if any operation in its batch has a conflict.

The EPaxos paper measurements did not capture the drawback of batching, as they measured batching only with the '0%' and '100%' loads, and batching has no effect on conflict rate for either of those workloads. Figure 11a shows that the mean latency for EPaxos is worse than Multi-Paxos for most of our workloads when batching is enabled (compare with Figure 7, where EPaxos mean latency is almost always lower than Multi-Paxos). Figure 11b shows more detail for a particular workload. With batching, the conflict rates are 6-16x higher than without batching. As a consequence of high conflict rates, batching causes EPaxos mean latency to be more than double that of both Multi-Paxos and EPaxos without batching.

6.6 Wide-Area Throughput

EPaxos throughput, like latency, is heavily impacted by operation interference. Figure 12 shows the maximum throughput EPaxos can sustain for each workload point, both with and without the optimization to bound dependency chains. Even with bounded dependency chains, throughput varies by more



(a) Improved **(b)** Unmodified

Figure 12: EPaxos throughput for a variety of workloads, both with and without bounded dependency chains. Throughput was determined by varying the Poisson rate parameter until latency increased without an increase in throughput. Multi-Paxos maximum throughput is 15,000 operations per second. For the improved version of EPaxos, Multi-Paxos throughput is equal to that of EPaxos with a Zipfian skew of .99 and 100% writes, which is why there is no 'MPaxos' line on Figure (a).

than a factor of two across the range of workloads. Without the optimization it varies by a factor of 10. Throughput for the improved version of EPaxos is at least as good as Multi-Paxos for all workloads, and more than double Multi-Paxos when there is little interference. Throughput for the unimproved version of EPaxos can be as much as 3x lower than Multi-Paxos.

Although our results show significantly worse throughput for EPaxos than the original evaluation, we are not confident that our throughput measurements (or those in the EPaxos paper) are a pure reflection of protocol differences rather than of confounding factors. When investigating the factors that limit server throughput, we observed several anomalies as servers neared saturation. Seemingly innocuous changes could have a large impact on throughput. For example, increasing the number of available cores caused throughput to drop. We believe that some of the issues may come from unexpected behavior of Go's thread scheduler. For example, for maximum throughput it is important that the thread implementing the EPaxos or Multi-Paxos protocol have exclusive access to a core. We suspect that Go occasionally deschedules that thread in order to run less-critical threads that listen for incoming messages.

Further, there are particular implementation details that may artificially limit throughput for both EPaxos and Multi-Paxos. For example, EPaxos has a thread that cycles through unexecuted instances, checking if they can be executed, which becomes increasingly expensive as the number of unexecuted instances increase, and could likely be optimized. In addition, we measured that Multi-Paxos disproportionately suffers from imbalance in the program's threading structure.

We experimented with wildly modified versions of the code that eliminated many of these compounding factors, but concluded that measuring the best possible throughput that is achievable with either protocol was tangential to the goal of our work. This would have required effectively rewriting the code, and we wanted to keep constant with the original EPaxos evaluation as many factors as possible.

Nonetheless, we felt that it was important to disclose our findings, as our results differ from those of the original EPaxos evaluation, and the compounding factors that we encountered apply to the original EPaxos evaluation as well as ours.

7 Related Work

A number of existing protocols exploit commutativity to provide fast replication. Other commutativity-based protocols may suffer from similar performance degradation to EPaxos in a WAN, but the EPaxos slow path can incur higher latency than that of most other protocols. We leave analysis of other protocols to future work. *Generalized Paxos* [16] allows clients to complete commutative operations in 1 WAN RTT. Generalized Paxos has a 3 RTT slow path because it reverts to a stable leader when operations interfere, and it also has a larger quorum size than EPaxos. *CURP* [24] completes commutative operations in 1 RTT by separating durability from ordering. CURP uses a stable leader, but has clients also issue their operations to 'witnesses', which keep track of a set of operations that have been seen but not yet ordered by the leader. If a client issues an operation that interferes with another that is unordered, witnesses reject that operation and it must take the slow path. Unlike EPaxos, CURP commutativity does not use ordering; even if all witnesses process an interfering operation in the same order, it will still conflict based on the presence of interference.

Bipartisan Paxos [30] (BPaxos) uses modularity to address scaling issues of optimized consensus protocols. BPaxos allows bottleneck modules to be scaled independently from the rest of the system, increasing throughput. The BPaxos evaluation has similar limitations to that of EPaxos; a fixed 'conflict percentage' is chosen instead of using a representative distribution, and they increase throughput by adding client threads. The protocol is evaluated in a local-area network (LAN), and the authors list poor performance in a WAN as a limitation.

TAPIR [31], *Janus* [22], and *MDCC* [14] combine concurrency control and consensus to commit commutative distributed transactions in 1 WAN RTT. Previously, replication and transaction protocols would each separately, and redundantly, ensure consistency. By only ensuring consistency once, these protocols decrease latency by 1 WAN RTT.

Speculative Paxos [25] and *Network-Ordered Paxos* [17] are consensus protocols that use ordering to complete operations in 1 RTT. Both use specialized networking hardware to serialize instances within the network, which is infeasible in a WAN.

Speculative PBFT [29] and and *Eve* [13] are replication protocols that optimistically execute operations before their ordering is confirmed, rolling back those that conflict. Speculative PBFT allows clients to speculatively execute based on a prelimary result while the leader replicates the command. However, the result cannot be externally visible before the client learns that it is committed. Eve allows for concurrent execution within a multi-core system.

PLATO [7] orders multi-cast messages in a LAN. A replica detects out-of-order packets by observing packet inter-arrival times. PLATO tags each message with a timestamp before which it should not be processed. This allows the packet to be revoked if another arrives that PLATO believes should have an earlier ordering. In addition to being insuitable for a leaderless protocol, PLATO's approach is speculative on the receiver, while our timestamps are determined by the sender and will be the same for each receiver.

CAESAR [5] and *Clock-RSM* [9] are consensus protocols that use synchronized clocks. CAESAR is a leaderless protocol that is similar to EPaxos, and uses synchronized clocks to allow instances to take the fast path even if the quorum disagrees on an instance's dependencies. In order to do this, CAESAR increases the quorum size by one node, which increases the minimum latency of operations. Clock-RSM uses timestamps from synchronized clocks to order instances. Instances are sorted by timestamp, and can complete when no unknown instance could have had an earlier timestamp. Because of this, a given instance depends on all other replicas communicating that they have no prior instances before it can complete. Much of the latency benefit of Clock-RSM is due to replicas broadcasting replies to all other replicas, which would also be possible for EPaxos.

Spanner [8], Google's globally distributed database, uses synchronized clocks to linearize transactions. This is at a higher level than the consensus layer, which uses Multi-Paxos.

On the correctness of Egalitarian Paxos [26] identifies an error in the EPaxos recovery procedure that breaks correctness. A potential fix for this issue is implemented in [27]. Our evaluation is concerned with the performance of normal operation.

8 Conclusion

In this paper we have shown how conclusions about a system can be biased by elements of the experimental methodology such as workload and metrics. Although we agree with the original evaluation that EPaxos reduces median latency compared to Multi-Paxos, we show that the percentage of operations that acheives this low latency varies greatly by application. Additionally, there is high variance in operation latency; for many workloads, a roughly 30% reduction in median latency can be at the expense of a more than 4x increase in 99th percentile latency. By covering a larger range of workloads, we also encountered issues that did not manifest in the original EPaxos paper, such as unbounded dependency chains, the cost of dependency delays (which can affect even the fast path), the latency penalty from batching, and the impact of conflicts on throughput.

We also presented TOQ, a novel technique that uses synchronized clocks to improve the performance of consensus protocols for geo-replication. We evaluated TOQ on EPaxos and demonstrated that it can reduce the conflict rate and mean latency of EPaxos without introducing additional latency.

9 Acknowledgements

Our work would not have been possible without the Egalitarian Paxos authors' transparency about their evaluation, both from the details in the body of their paper and the availability of their source code. We thank the authors of EPaxos, particularly Iulian Moraru, for their responsiveness to questions we had about the paper, protocol, their experiments, and our findings.

This work was supported by the industrial affiliates of the Stanford Platform Lab. Our experiments were run on Google Cloud with the support of a Google Cloud Platform Education Grant. We thank our shepherd, Shuai Mu, and the anonymous reviewers for their helpful comments on the paper.

References

[1] Zookeeper api. https://zookeeper.apache.org/doc/r3.4.6/api/org/apache/zookeeper/ZooKeeper.html, January 2016.

[2] Redis api. https://redis.io/commands, May 2020.

[3] T. Armstrong. Linkbench code base. https://github.com/facebookarchive/linkbench, November 2012.

[4] T. G. Armstrong, V. Ponnekanti, D. Borthakur, and M. Callaghan. Linkbench: A database benchmark based on the facebook social graph. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, SIGMOD '13, page 1185–1196, New York, NY, USA, 2013. Association for Computing Machinery.

[5] B. Arun, S. Peluso, R. Palmieri, G. Losa, and B. Ravindran. Speeding up consensus by chasing fast decisions. In *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 49–60, 2017.

[6] B. Atikoglu, Y. Xu, E. Frachtenberg, S. Jiang, and M. Paleczny. Workload analysis of a large-scale key-value store. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems*, SIGMETRICS '12, page 53–64, New York, NY, USA, 2012. Association for Computing Machinery.

[7] M. Balakrishnan, K. Birman, and A. Phanishayee. Plato: Predictive latency-aware total ordering. In *2006 25th IEEE Symposium on Reliable Distributed Systems (SRDS'06)*, pages 175–188, 2006.

[8] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford. Spanner: Google's globally-distributed database. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*, pages 261–264, Hollywood, CA, 2012. USENIX Association.

[9] J. Du, D. Sciascia, S. Elnikety, W. Zwaenepoel, and F. Pedone. Clock-rsm: Low-latency inter-datacenter state machine replication using loosely synchronized physical clocks. In *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 343–354, 2014.

[10] Y. Geng, S. Liu, Z. Yin, A. Naik, B. Prabhakar, M. Rosunblum, and A. Vahdat. Exploiting a natural network effect for scalable, fine-grained clock synchronization. In *Proceedings of the 15th USENIX Conference on Networked Systems Design and Implementation*, NSDI'18, page 81–94, USA, 2018. USENIX Association.

[11] J. Gray, P. Sundaresan, S. Englert, K. Baclawski, and P. J. Weinberger. Quickly generating billion-record synthetic databases. In *Proceedings of the 1994 ACM SIGMOD International Conference on Management of Data*, SIGMOD '94, page 243–252, New York, NY, USA, 1994. Association for Computing Machinery.

[12] J. Gray, P. Sundaresan, S. Englert, K. Baclawski, and P. J. Weinberger. Quickly generating billion-record synthetic databases. *SIGMOD Rec.*, 23(2):243–252, May 1994.

[13] M. Kapritsos, Y. Wang, V. Quema, A. Clement, L. Alvisi, and M. Dahlin. All about eve: Execute-verify replication for multi-core servers. In *10th USENIX Symposium*

on Operating Systems Design and Implementation (OSDI 12), pages 237–250, Hollywood, CA, Oct. 2012. USENIX Association.

[14] T. Kraska, G. Pang, M. J. Franklin, S. Madden, and A. Fekete. Mdcc: Multi-data center consistency. In *Proceedings of the 8th ACM European Conference on Computer Systems*, EuroSys '13, page 113–126, New York, NY, USA, 2013. Association for Computing Machinery.

[15] L. Lamport. The part-time parliament. *ACM Trans. Comput. Syst.*, 16(2):133–169, May 1998.

[16] L. Lamport. Generalized consensus and paxos. Technical Report MSR-TR-2005-33, March 2005.

[17] J. Li, E. Michael, N. K. Sharma, A. Szekeres, and D. R. K. Ports. Just say NO to paxos overhead: Replacing consensus with network ordering. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 467–483, Savannah, GA, Nov. 2016. USENIX Association.

[18] I. Moraru. There is more consensus in egalitarian parliaments. https://www.youtube.com/watch?v=KxoWlUZNKn8.

[19] I. Moraru, D. G. Andersen, and M. Kaminsky. A proof of correctness for egalitarian paxos. 2012.

[20] I. Moraru, D. G. Andersen, and M. Kaminsky. Epaxos code base. https://github.com/efficient/epaxos, August 2013.

[21] I. Moraru, D. G. Andersen, and M. Kaminsky. There is more consensus in egalitarian parliaments. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, page 358–372, New York, NY, USA, 2013. Association for Computing Machinery.

[22] S. Mu, L. Nelson, W. Lloyd, and J. Li. Consolidating concurrency control and consensus for commits under conflicts. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, OSDI'16, page 517–532, USA, 2016. USENIX Association.

[23] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 305–319, Philadelphia, PA, June 2014. USENIX Association.

[24] S. J. Park and J. Ousterhout. Exploiting commutativity for practical fast replication. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 47–64, Boston, MA, Feb. 2019. USENIX Association.

[25] D. R. K. Ports, J. Li, V. Liu, N. K. Sharma, and A. Krishnamurthy. Designing distributed systems using approximate synchrony in data center networks. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pages 43–57, Oakland, CA, May 2015. USENIX Association.

[26] P. Sutra. On the correctness of egalitarian paxos. *CoRR*, abs/1906.10917, 2019.

[27] P. Sutra. On the correctness of egalitarian paxos code base. https://github.com/otrack/epaxos, October 2019.

[28] S. Tollman. Epaxos revisited code base. https://github.com/platformlab/epaxos, March 2021.

[29] B. Wester, J. Cowling, E. B. Nightingale, P. M. Chen, J. Flinn, and B. Liskov. Tolerating latency in replicated state machines through client speculation. In *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation*, NSDI'09, page 245–260, USA, 2009. USENIX Association.

[30] M. Whittaker, Giridharanil, A. Szekeres, J. M. Hellerstein, and I. Stoica. Bipartisan paxos: A modular state machine replication protocol. *arXiv preprint arXiv:2003.00331*, 2020.

[31] I. Zhang, N. K. Sharma, A. Szekeres, A. Krishnamurthy, and D. R. K. Ports. Building consistent transactions with inconsistent replication. In *Proceedings of the 25th Symposium on Operating Systems Principles*, SOSP '15, page 263–278, New York, NY, USA, 2015. Association for Computing Machinery.

A Appendix

A.1 Upper Bound on Execution Latency

With the improvement to EPaxos described in Section 3, execution latency is bounded at roughly 3 WAN RTTs. Let's say that replica A originates instance A.*i* and has transitive dependency B.*i* from replica B. In order for B.*i* to have an equal or lower sequence number than A.*i*, or not have A.*i* as a dependency, some other replica C in B's fast-path quorum must learn about B.*i* before A.*i*.

In the worst case, C will learn about B.*i* an instant before C learns about A.*i*, B.*i* will take the slow path, and B.*i*'s commit must propagate to A. Thus, the latency bound is the sum of:

- 1 message delay: the one-way delay from A to C, which is A.*i*'s PreAccept message.
- <2 message delays: the remainder of the one-way delay from B to the other replicas in its quorum + the one-way delay for their responses. (This will be 1 message delay if C is the furthest replica in B's quorum, and always less than 2 message delays. For our topology, this is always 1 message delay.)
- 2 message delays: an RTT between B and its quorum, for the Accept phase.
- 1 message delay: the one-way delay from B to A, for B.*i*'s commit to propagate to A.

As an example, we can compute the upper bound on execution latency for Japan in our test topology. Figure 5 contains the RTTs between replicas in the topology, and we estimate that the one-way delay is half of RTT. The worst case for Japan (JP) is that an instance from London (EU) arrives at Oregon (OR) an instant before Japan's instance and takes the slow path. The maxmimum execution latency for the instance from Japan will be 45.5ms (one-way delay from JP to OR, for JP's PreAccept) + 64.5ms (one-way delay from OR to EU for OR's PreAccept reply) + 129ms (RTT between EU and OR, for the Accept phase) + 110.5ms (EU's commit propagation to JP) = 349.5ms. This is consistent with the maximum execution latency we measured for clients in Japan in our experiments, which was 349.9ms.

A.2 Impact of Thrifty on Conflict Rate

The original EPaxos evaluation used an optimization called *thrifty*, in which an originator only sends PreAccept and Accept messages to the necessary quorum. Although Enabling thrifty increases throughput for both EPaxos and Multi-Paxos by about 15%, enabling thrifty increases EPaxos conflict rates. With the thrifty optimization, some nodes do not become aware of an instance until it has committed. When some nodes know about an instance but others don't, there is increased potential for discrepancies in PreAcceptReplies, leading to increased conflicts.

Figure 13 provides an example of this. In the figure, replica E sends a PreAccept message for instance E.0 before replica A sends a PreAccept message for A.0. With thrifty enabled, only one of the two replicas in A's fast-path quorum receives the PreAccept for E.0 before the PreAccept for A.0. This causes

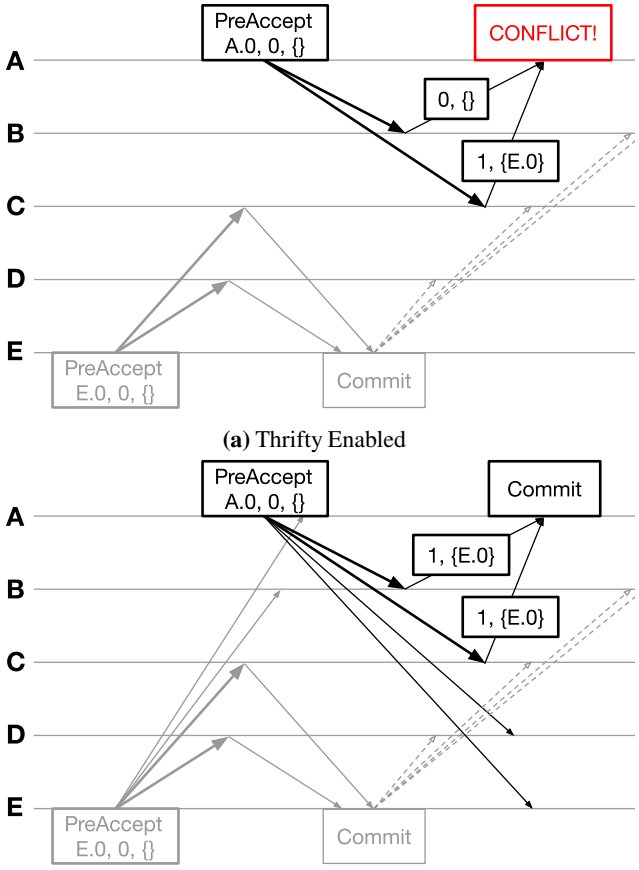


Figure 13: EPaxos PreAccept phase example, with and without *thrifty*. Replicas A and E originate interfering instances A.0 and E.0 concurrently. A's fast-path quorum consists of replicas B and C, and E's fast-path quorum consists of replicas C and D. With thrifty enabled, the responses from B and C differ for A.0. With thrifty disabled, replica B receives the PreAccept for E.0 before A.0, avoiding conflict.

A.0 to take the slow path. With thrifty disabled, both replicas in A's fast-path quorum receive the PreAccept for E.0 before that for A.0, so A.0 can commit on the fast path.

In Figure 14, we demonstrate experimentally that thrifty increases EPaxos conflict rates. Conflict rates are higher with thrifty enabled for all three workloads in the figure, which generally leads to higher latency.

A.3 Impact of Throughput and Workload Size on Conflict Rate

When comparing EPaxos to a variety of skews and write ratios in Section 6, we held throughput and the number of unique keys in the workload constant. However, both of these variables also impact operation interference.

Figure 15 shows that EPaxos conflict rate, mean latency, and 99th percentile latency increase as throughput increases. Higher throughput corresponds to having a higher number of concurrent operations, creating more opportunities for interference. EPaxos 99th percentile latency plateaus once more

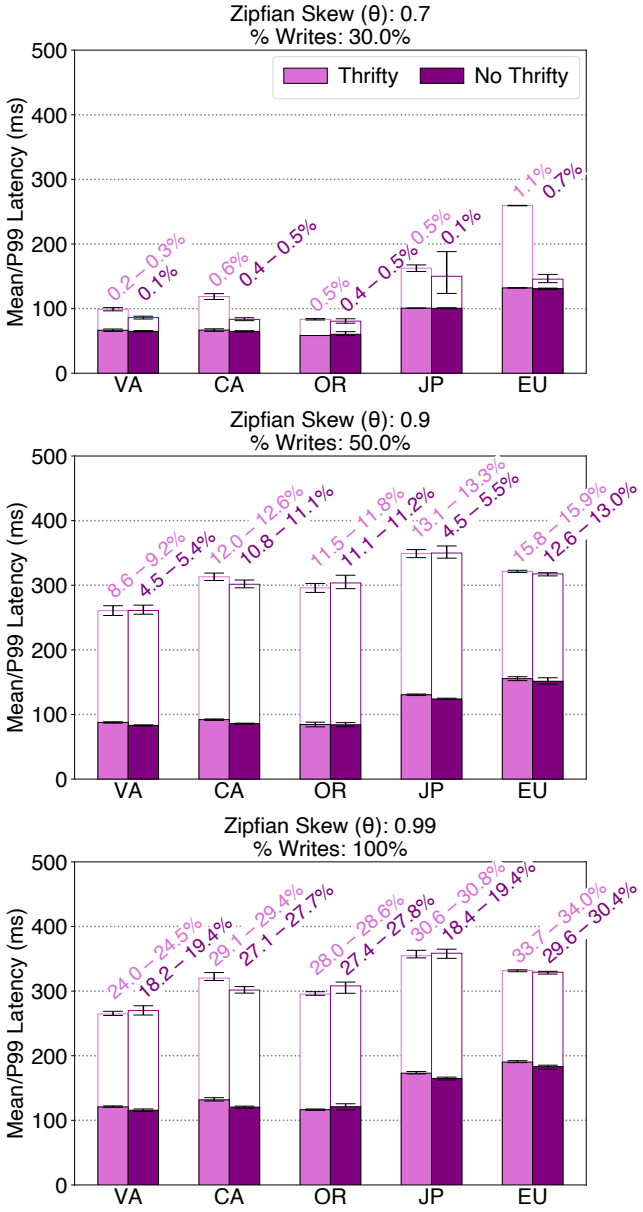


Figure 14: Comparing EPaxos Mean and 99th percentile execution latency with and without *thrifty* enabled for three different workloads. The experiments for this graph where run with the same methodology as Figure 7. Error bars and conflict rates are shown as in Figure 6.

than 1% of operations hit the upper bound on execution latency described in Appendix A.1.

Figure 16 shows that EPaxos conflict rate, mean latency, and 99th percentile latency decrease as the number of unique keys in the workload increases. Although our latency analyses showcase a variety of skew factors, our analyses only display measurements for a single number of unique keys. The trends would be the same with a different number of unique keys, but the measurements for each data point would be different.

A.4 Additional TOQ Graphs

In Section 6.3 and with Figure 9, we demonstrated that TOQ reduced conflict rate and, consequently, latency for clients in Oregon (OR). Figure 17 shows that TOQ reduces conflict rate for clients in all locations. Figure 18 shows that TOQ reduces Mean latency for all client locations, except for clients in Japan with quorum union sync. Figure 19 shows that TOQ reduces 99th percentile latency for all client locations. Mean latency increases for Japan with quorum union sync because the quorum union contains VA, which is significantly further from JP than its quorum, adding about 24ms to the minimum operation latency for clients in Japan. Although Mean latency increases for clients in Japan with quorum union sync, 99th percentile latency still decreases, due to decreased conflict rate.

Figure 20 demonstrates the benefits and tradeoffs of TOQ for two specific workloads. For both workloads, TOQ with a sync group of all replicas completely eliminates conflict. However, the mean latency for the 'all' sync group is higher than without TOQ because its minimum latency is higher. The quorum union sync group decreases conflict rate for all locations, and latency for most locations, but latency for JP is higher because the minimum latency is higher. The quorum sync group does not decrease conflict rate to the same extent as the 'all' or quorum union sync groups, but it purely decreases latency because its minimum latency is the same as without TOQ.

For the top workload in Figure 20, TOQ primarly decreases 99th percentile latency. Without TOQ, more than 1% of operations are at the latency bound described in Section A.1. TOQ reduces command interference enough that fewer than 1% of operations are at the bound. Because conflict rates without TOQ are fairly low, the impact of TOQ on mean latency for the top workload is minimal.

For the bottom workload in Figure 20, conflict rates are high enough that TOQ primarly decreases mean latency. There is a minimal impact on 99th percentile latency for all sync groups other than the 'all' sync group because, even with the quorum union sync group, more than 1% of operations are at the latency bound.

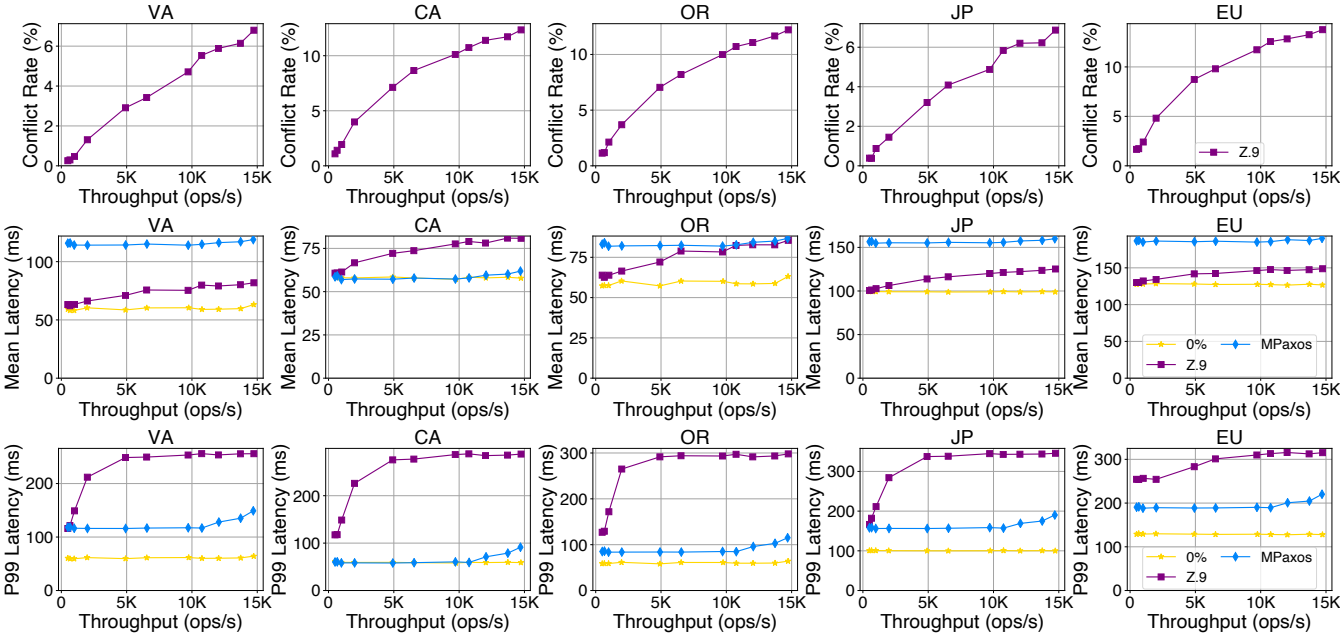


Figure 15: Conflict rate, mean latency, and 99th percentile latency as a function of throughput. To increase throughput, we varied the Poisson rate parameter. Maximum throughput for this the graph is the maximum that produces reasonable behavior for all protocols and experiments. 50% of client operations are writes, and the Zipfian distribution has 1 million unique keys.

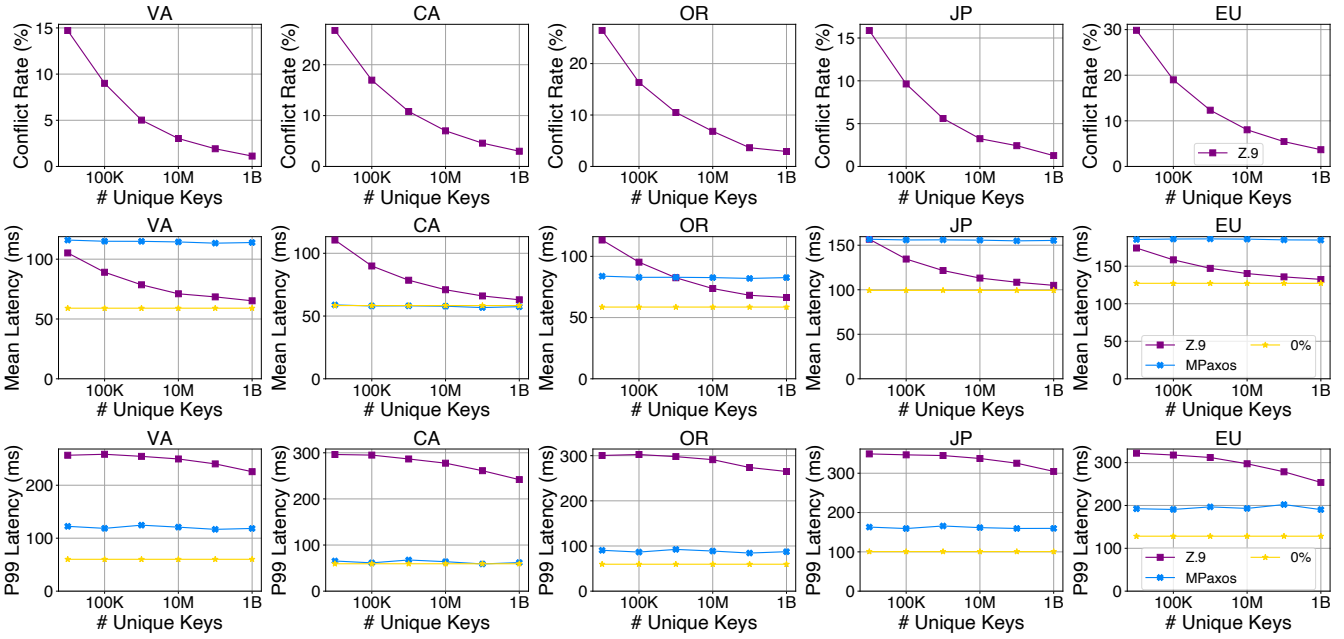


Figure 16: Conflict rate, mean latency, and 99th percentile latency as a function of the number of unique keys in the workload. All experiments were run at a throughput of 11,000 operations per second, which is 80% of maximum throughput for Multi-Paxos. 50% of client operations are writes.

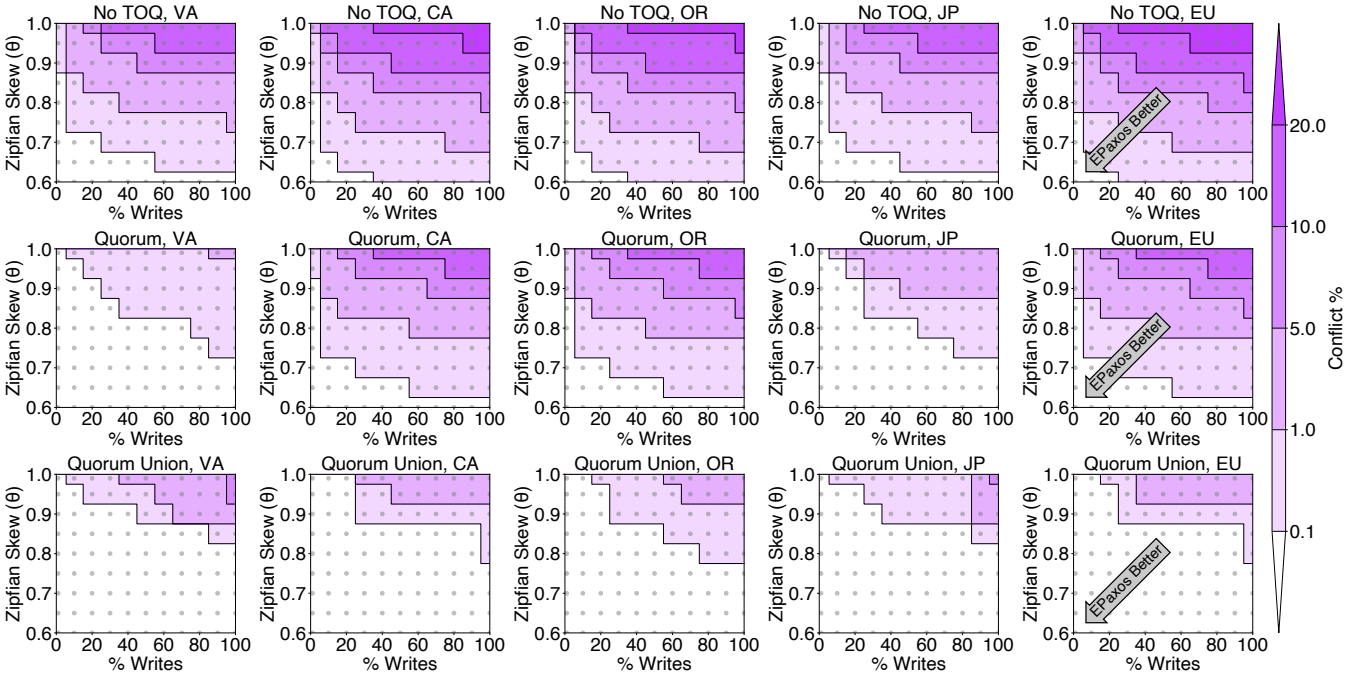


Figure 17: Comparing conflict rate for EPaxos without TOQ, EPaxos with TOQ to a quorum sync group, and EPaxos with TOQ to a quorum union sync group. The quorum union consists of Virginia, California, and Oregon. Other than TOQ, the experiments for this graph were run with the same methodology as Figure 7.

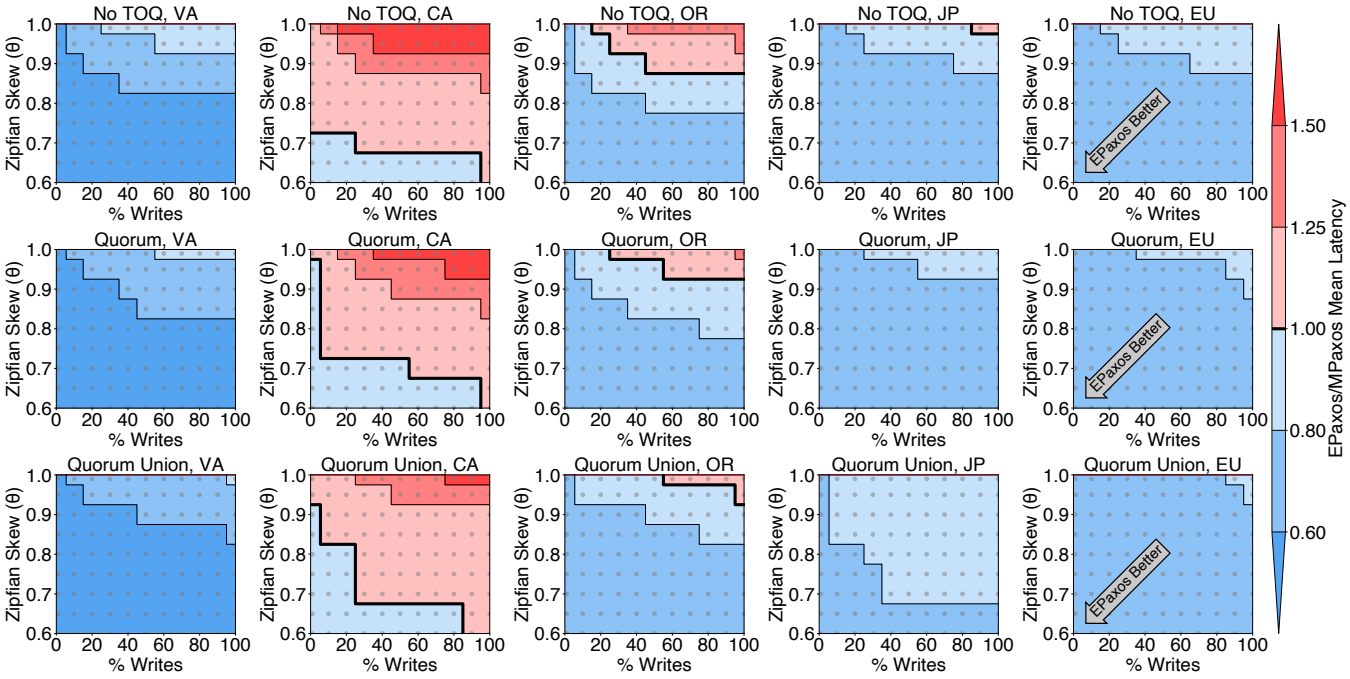


Figure 18: Comparing mean latency for EPaxos without TOQ, EPaxos with TOQ to a quorum sync group, and EPaxos with TOQ to a quorum union sync group. The quorum union consists of Virginia, California, and Oregon. Other than TOQ, the experiments for this graph where run with the same methodology as Figure 7.

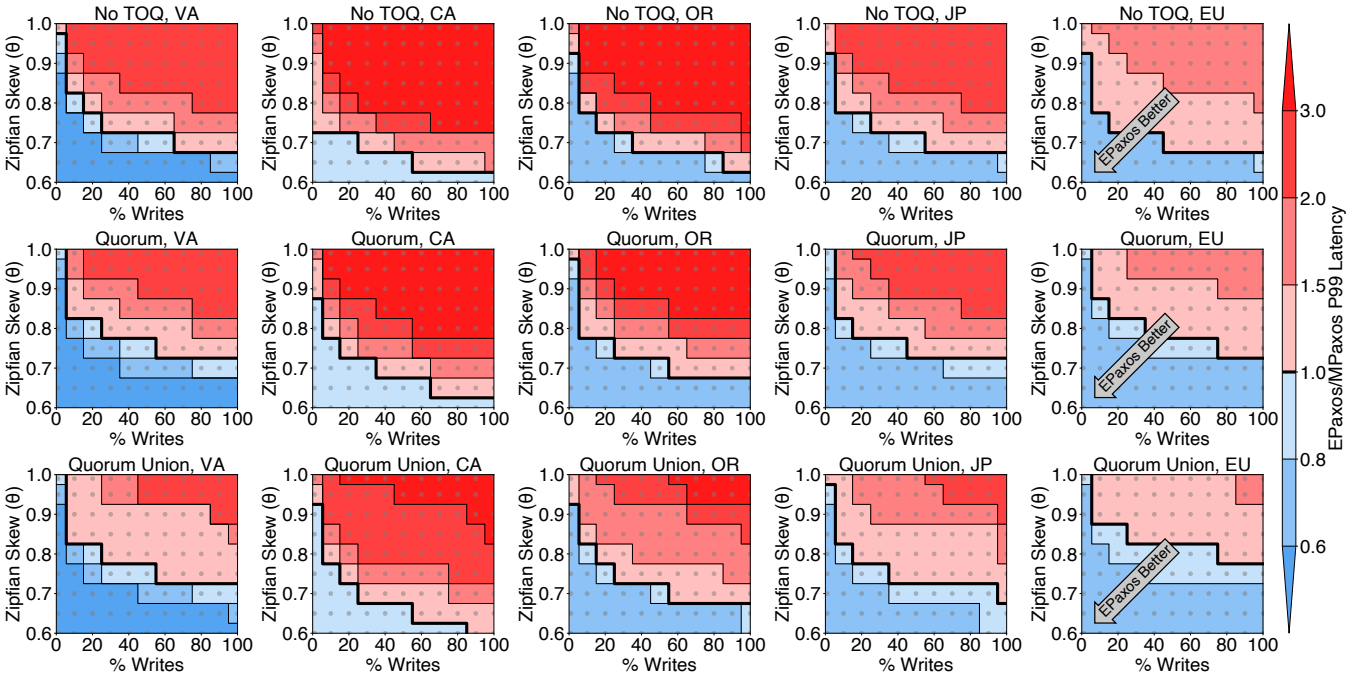


Figure 19: Comparing 99th percentile latency for EPaxos without TOQ, EPaxos with TOQ to a quorum sync group, and EPaxos with TOQ to a quorum union sync group. The quorum union consists of Virginia, California, and Oregon. Other than TOQ, the experiments for this graph where run with the same methodology as Figure 7.

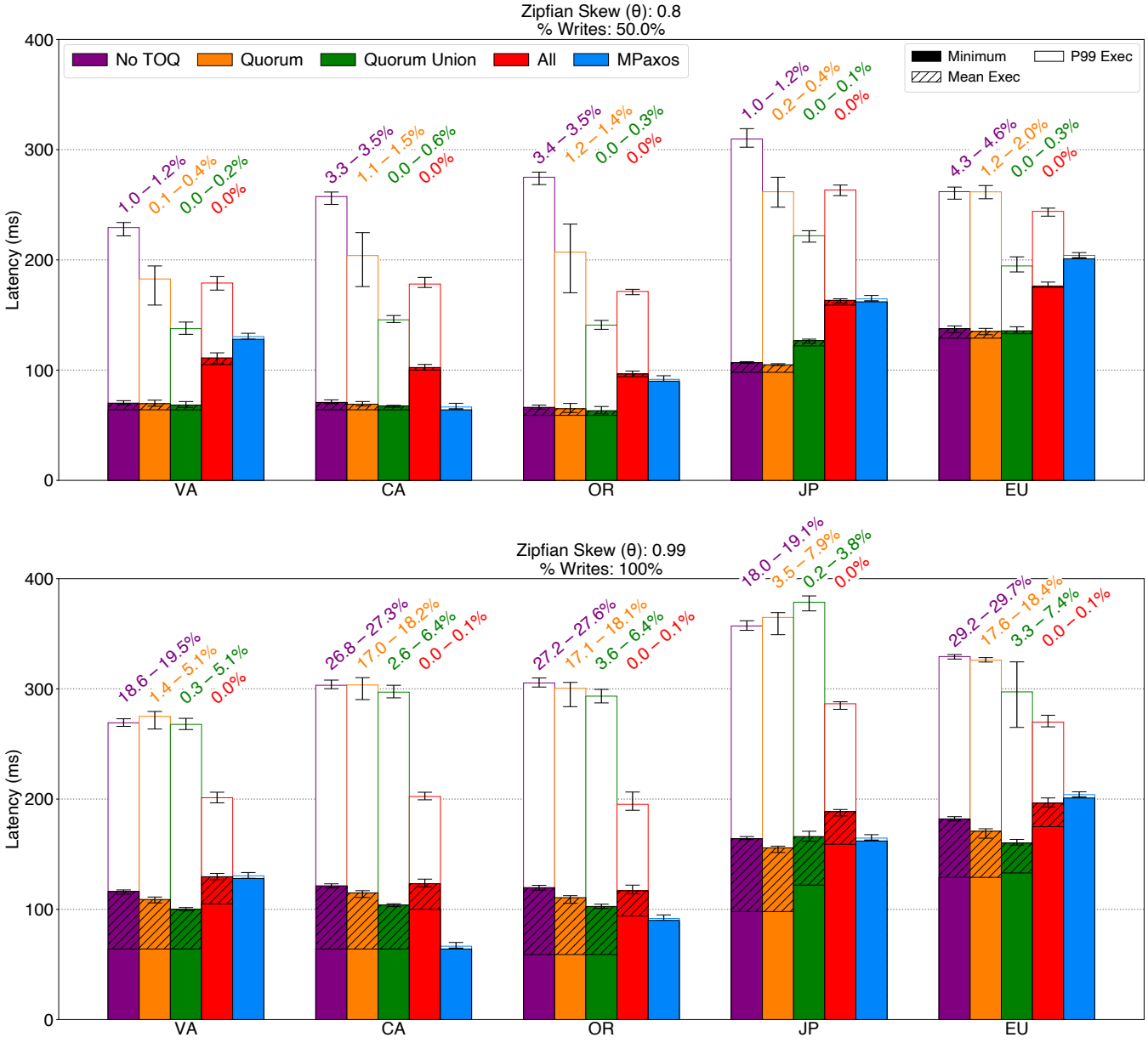


Figure 20: Comparing Mean and 99th percentile latency for EPaxos with three different TOQ sync groups. The top graph is a medium-conflict workload, and the bottom is a high-conflict workload. 'Minimum' refers to the minimum possible execution latency for the protocol and sync group. The quorum union consists of Virginia, California, and Oregon. Other than TOQ, the experiments for this graph where run with the same methodology as Figure 7. Error bars and conflict rates are shown as in Figure 6.